



HASHICORP TERRAFORM

Introduction to Terraform

Infrastructure as Code, Declarative Provisioning, and How Terraform Compares

What is Infrastructure as Code?

Traditional, manual provisioning vs. defining infrastructure in code

BEFORE: Traditional Infrastructure



Time-consuming

Manual server setup takes significant effort



Error-prone

Manual configuration invites inconsistency



Hard to scale

Managing thousands of servers by hand is impractical



Hard to track changes

Difficult to roll back to a prior configuration

AFTER: Infrastructure as Code

Define infrastructure in version-controlled code; a tool like Terraform provisions it automatically.



Automated



Consistent



Reproducible



Scalable

How it works: define config → run an IaC tool → resources are provisioned & managed automatically.

What is Terraform?

An open-source **Infrastructure as Code tool by HashiCorp** that defines, deploys, and manages infrastructure across cloud providers using the declarative language HCL.



Multi-Cloud Support

Works with AWS, Azure, GCP, Kubernetes, and on-prem



Declarative Approach

Define desired state; Terraform figures out how to reach it



Immutable Infrastructure

Ensures consistency by replacing, not modifying, resources



State Management

Tracks infrastructure changes in a state file



Modular & Scalable

Reusable modules manage large-scale infrastructure

Terraform vs. Ansible

Provisioning infrastructure vs. configuring it

Feature	Terraform	Ansible
Approach	Declarative	Imperative
Use Case	Provision infrastructure	Configure infrastructure
State Management	Maintains a state file	No state file
Cloud Agnostic	Yes	Yes
Example	Create an EC2 instance	Install Nginx on an EC2 instance



Terraform creates infrastructure, while **Ansible** configures it — many teams use both together.

Terraform vs. CloudFormation

Multi-cloud flexibility vs. deep AWS integration

Feature	Terraform	CloudFormation
Cloud Support	Multi-cloud	AWS-only
Language	HCL	JSON / YAML
State Management	Uses a state file	Managed by AWS
Modularity	Strong support for modules	Supports nested stacks
Speed	Faster	Slower



Terraform is more flexible and multi-cloud, while **CloudFormation** is tightly integrated with AWS.

Terraform vs. Pulumi

Configuration language vs. general-purpose programming languages

Feature	Terraform	Pulumi
Language	HCL	Python, Go, TypeScript
Cloud Support	Multi-cloud	Multi-cloud
State Management	Uses state file	State managed via Pulumi Service
Learning Curve	Easier for beginners	Harder due to programming languages



Pulumi lets you use real programming languages, but **Terraform** stays simpler for beginners.

Where Can You Use Terraform?

Terraform is highly versatile across environments and pipelines



Cloud Infrastructure

Provision VMs, databases, networking (AWS, Azure, GCP)



Kubernetes

Manage Kubernetes clusters and deployments



On-Prem

Automate VMware, OpenStack, or bare-metal infrastructure



Multi-Cloud

Deploy the same infrastructure across multiple providers



CI/CD Pipelines

Automate infrastructure deployment in GitLab CI/CD



Security & Compliance

Define security policies and enforce compliance

Hands-On: Same Infrastructure, Two Tools

Deploying an EC2 instance — Step 1: Terraform



main.tf

```
provider "aws" {  
  region = "us-east-1"  
}  
  
resource "aws_instance" "example" {  
  ami           = "ami-0c55b159cbfafa1f0"  
  instance_type = "t2.micro"  
  
  tags = {  
    Name = "Terraform-Instance"  
  }  
}
```

Apply it:

```
$ terraform init  
$ terraform apply -auto-approve
```



What happens

- Declares the AWS provider and target region
- Defines one EC2 resource: a t2.micro Amazon Linux instance
- terraform init downloads the AWS provider plugin
- terraform apply provisions the instance and writes state to a .tfstate file

Hands-On: Same Infrastructure, Two Tools

Deploying an EC2 instance — Step 2: AWS CloudFormation



ec2.yaml

```
AWSTemplateFormatVersion: "2010-09-09"
```

```
Resources:
```

```
  MyEC2Instance:
```

```
    Type: AWS::EC2::Instance
```

```
    Properties:
```

```
      ImageId: "ami-0c55b159cbfafa1f0"
```

```
      InstanceType: "t2.micro"
```

```
      Tags:
```

```
        - Key: Name
```

```
          Value: CloudFormation-Instance
```

Apply it:

```
$ aws cloudformation create-stack \  
  --stack-name MyEC2Stack \  
  --template-body file://ec2.yaml
```



What happens

- Declares a single `AWS::EC2::Instance` resource in YAML
- Same AMI and instance type as the Terraform example
- `create-stack` submits the template directly to AWS
- AWS manages the stack's state internally — no local state file

Comparing the Two Deployments

Same EC2 instance, Terraform vs. CloudFormation — head to head

Feature	Terraform	CloudFormation
Multi-Cloud Support	Yes	AWS-only
State Management	Uses a .tfstate file	Managed by AWS
Syntax	HCL (easier to read)	JSON / YAML (more complex)
Execution Speed	Faster	Slower
Modularity	Strong support	Supports nested stacks



Conclusion: **Terraform** is preferred for multi-cloud and flexibility, while **CloudFormation** is best if you're AWS-exclusive.

Key Takeaways



IaC turns manual, error-prone provisioning into automated, versioned, repeatable code.



Terraform is a declarative, multi-cloud tool using HCL, with state tracked in a .tfstate file.



Terraform provisions infrastructure; Ansible configures it — they pair well together.



Versus CloudFormation and Pulumi, Terraform trades some native integration and language flexibility for portability and simplicity.

Next up: writing your first Terraform module